Exercise 2
1 Data Set, 1 Model, and 6 Training Algorithms

How to approach the tasks on this exercise sheet?

Work in teams! Why? Because it hones your interpersonal communication skills! Those will always be important, even in the era of ever stronger AI.

Be ambitious and try to solve the following problems yourself. If you work with AI assistance, you don't have to code everything yourself. But try to figure out what needs to be coded so that you can provide specific instructions for your AI companion.

If your main goal is to learn to cooperate with a capable AI companion, then consider the following questions for your interactions:

- `How could we approach this task ?`
- `Why should we consider this approach ?`
- `Are there any other or better approaches ?`

Once you have solved a task together with an AI, you might want to ask these questions:

- `What are the main takeaways from this task ?`
- `Why would our instructor have set this task ?`
- `What could he have wanted us to learn by solving it ?`

For this particular exercise sheet, also think about the following overarching question:

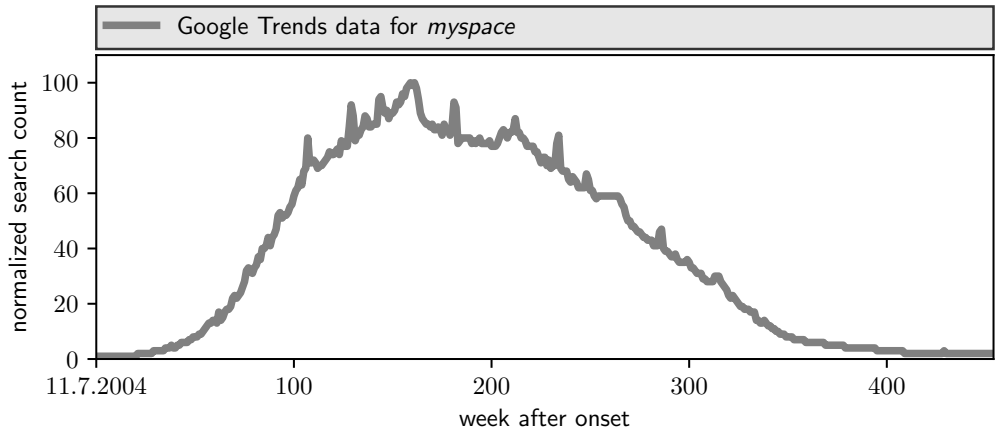
- `What may have been our instructor's point in asking us to fit one model to one (toy) data set using six different approaches ?`

Contents

Problem 2.1: setting the stage 2
Problem 2.2: fitting a Weibull to a histogram (part 1) 6
Problem 2.3: fitting a Weibull to a histogram (part 2) 9
Problem 2.4: fitting a Weibull to a histogram (part 3) 11
Problem 2.5: fitting a Weibull to a distribution (part 1) 14
Problem 2.6: fitting a Weibull to a distribution (part 2) 17
Problem 2.7: solving differential equations 20
Problem 2.8: what about neural networks? 22
References 23

Problem 2.1: setting the stage

In the early to mid 2010s, your instructor used machine learning techniques for data analysis in the field of *Web science* which researches how people interact with Web technologies such as social media [1], [2], [3], [4]. Back in March 2013, he retrieved a data set from Google Trends which, when pre-processed and plotted, looks like this:



What this figure shows is a *search frequency time series* related to the query term *myspace*. (Does anybody still know what *myspace* was?) Put differently, the graph in this figure indicates how often people across the world googled for *myspace*. Put differently again, it indicates how global interest in *myspace* measured in terms of Google searches evolved over time.

There are certain noteworthy details about our figure. Observe that its x-axis is labeled "week after onset". Indeed, back in 2013, the Google Trends team still provided weekly search statistics.

The *onset* of our time series coincides with the second week of July 2004 and marks the point in time when the first ever Google search for *myspace* happened. The length of our time series covers a period of slightly more than 400 weeks after onset. During this time, **global interest in *myspace* first grew quickly, then reached a peak, and afterwards saw a period of prolonged steady decline**.

Further note that the y-axis of our figure is labeled "normalized search counts". Indeed, the Google Trends team does not disclose absolute numbers. Instead, they normalize their publicly provided data such that the highest ever number of searches for a term gets mapped to 100. In other words, they only provide relative search counts or *search frequencies*.

From the point of view of keeping business secrets, this restriction to relative counts is understandable; from the point of view of data modeling and analysis, it may create headaches which you will have to address below. On the plus side, the peculiar nature of this kind of data also creates plenty of learning opportunities …

Task 2.1.1

The above *myspace* data can be found in the `csv` file `dataMyspace201303.csv`. Implement `Python` / `numpy` code that reads its second column into a 1D `numpy` array. Remove all leading zeros from this array, and store the n remaining relative search *count* values in an array

$$\boldsymbol{c} = [c_1, c_2, ..., c_n] \ . \tag{1}$$

Furthermore, create a 1D `numpy` array of n *time* steps

$$\boldsymbol{t} = [1, 2, 3, ..., n] \ . \tag{2}$$



Note
Yes, the entries of $\boldsymbol{t}$ are deliberately chosen to range from 1 up to n. Make sure that your code adheres to this choice.



Tip
Below, you will need to run these data pre-processing steps more than once. It may thus be a good idea to wrap them in a function `load_myspace_data()`.

Next, implement `matplotlib` code (which you should also wrap in a reusable function) that plots the data in $\boldsymbol{c}$ against the data in $\boldsymbol{t}$. While your plots do not have to be as fancy as the one above, they should at least resemble it.

Below, there will (of course) be further tasks. But before we get to them, we better look at some background information to get to grips with what they will be all about ...

Background Info

Note that we are dealing with **histogram** data, namely with a distribution of discrete search *counts* c_t over discrete *times* t.

Also note that we can always think of discrete count data as the result of sampling some *latent* (i.e. unknown to us), continuous probability density.

In our lectures, we frequently work with Gaussian distributions to model latent densities because they are exceptionally easy to handle. However, ease of use should never be our primary objective when modeling real life data. Instead, **models must be plausible and capture crucial characteristics of the data we are working with**.

Hence, consider this: The above figure clearly shows that searches for *myspace* were not Gaussian distributed over time. Again, interest first grew quickly, reached some peak, and then steadily declined. In other words, while a Gaussian is symmetric, the distribution

in the figure is positively skewed (its right tail is longer than its left tail). Modeling the temporal evolution of *myspace* searches in terms of a Gaussian would therefore be inappropriate.

A much better model for our data is the **Weibull distribution**. It is defined for $t \in [0, \infty)$ and its *probability density function* and *cumulative density function* are given by

$$f(t \mid \alpha, \beta) = \frac{\alpha}{\beta}\left(\frac{t}{\beta}\right)^{\alpha-1} \exp\left[-\left(\frac{t}{\beta}\right)^{\alpha}\right] \tag{3}$$

and

$$F(t \mid \alpha, \beta) = 1 - \exp\left[-\left(\frac{t}{\beta}\right)^{\alpha}\right] \tag{4}$$

respectively; the two parameters $\alpha, \beta \in \mathbb{R}_+$ are positive real numbers which determine the **shape** and **scale** of a Weibull distribution.

Mini Exercise

Given (3) and (4), show that $f = \frac{\mathrm{d}F}{\mathrm{d}t}$.

More Background Info

But why would we consider the Weibull as a (plausible) model for the for the temporal distribution of *myspace* search frequencies? Because it implicitly encodes growth-and-decline dynamics or rise-and-fall processes!

To see why or how this is, we note that we may rewrite (4) as $\exp[-(t/\beta)^{\alpha}] = 1 - F(t)$ which we can then plug into (3) to obtain

$$\begin{aligned} f(t) &= \frac{\alpha}{\beta}\left(\frac{t}{\beta}\right)^{\alpha-1}(1 - F(t)) \\ &= \frac{\alpha}{\beta}\left(\frac{t}{\beta}\right)^{\alpha-1} - \frac{\alpha}{\beta}\left(\frac{t}{\beta}\right)^{\alpha-1} F(t)\ . \end{aligned} \tag{5}$$

Hence, the Weibull sneakily involves a difference between a growth- and a decline term which are both polynomial in t. Moreover, since the decline term also depends on $F(t)$ which itself grows monotonously, we find that the longer a process is running whose dynamics we characterize by a Weibull distribution, the quicker the decline.

Below, you will see that the Weibull is not necessarily the best model for how public interest in social media sites grows and declines. The real contender in this arena is the **generalized Gamma distribution** [3], [4] Alas, since this more powerful model is

slightly too cumbersome for us to work with in these exercises, we focus on the Weibull for simplicity …

Task 2.1.2

Code two properly parameterized `numpy` functions `weibull_pdf()` and `weibull_cdf()` which implement and return the right hand sides of (3) and (4).



Note
Do *not* work with the Weibull implementations provided in `scipy.stats`. Those are somewhat lobotomized versions of the functions in (3) and (4) and nobody knows why the `scipy` developers opted to ignore the real Weibull distribution …

If you feel uneasy about the notions of shape and scale of a probability distribution, simply `import matplotlib.pyplot as plt` and run the following snippet. Its graphical outputs should clarify what shape and scale of a Weibull are all about.

```
ts = np.linspace(0, 10, 100)

for alpha in [0.5, 1.0, 1.5, 2.0, 2.5]:
    plt.plot(ts, weibull_pdf(ts, alpha, 2.0), '-')
plt.show()

for beta in [0.5, 1, 2, 3, 4]:
    plt.plot(ts, weibull_pdf(ts, 2.5, beta),  '-')
plt.show()
```

Problem 2.2: fitting a Weibull to a histogram (part 1)

Given what you accomplished for the previous problem, your next task is to fit a Weibull probability density $f(t \mid \alpha, \beta)$ to the *myspace* data (t, c_t) such that you get

$$c_t \sim f(t \mid \alpha, \beta) \tag{6}$$

up to a constant scaling factor.

One would think that `scipy.stats` or `scipy.optimize` would provide ready-made tools or functions for this task. Alas, neither one ships with tools that fit *continuous* densities to *discrete* and potentially *truncated* histograms. You therefore have to implement them for yourself …

We already know about maximum likelihood estimation of the parameters of Gaussian distributions. Alas, maximum likelihood estimation of the parameters of a Weibull distribution is much more involved. To see why, we next "naively" transfer what we learned about the Gaussian setting to our current Weibull setting.

Given a *data sample* $D = \{x_i\}_{i=1}^N$, the log-likelihood for the parameters of the Weibull distribution is

$$\mathcal{L}(\alpha, \beta \mid D) = N \cdot (\log \alpha - \alpha \log \beta) + (\alpha - 1) \cdot \sum_i \log x_i - \sum_i \left(\frac{x_i}{\beta}\right)^\alpha . \tag{7}$$

Now, looking at our trusted recipe of estimating optimal parameters $\hat{\alpha}$ and $\hat{\beta}$ by solving

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \alpha} &= 0 \\ \frac{\partial \mathcal{L}}{\partial \beta} &= 0 \end{aligned} \tag{8}$$

we quickly realize that this system of coupled partial differential equations does not admit a closed form solution.

However, since we also already know about Newton's method for parameter optimization, we may guess a solution and then iteratively refine it via the following iterative updates

$$\begin{bmatrix} \alpha \\ \beta \end{bmatrix} \leftarrow \begin{bmatrix} \alpha \\ \beta \end{bmatrix} - \begin{bmatrix} \frac{\partial^2 \mathcal{L}}{\partial \alpha^2} & \frac{\partial^2 \mathcal{L}}{\partial \alpha \partial \beta} \\ \frac{\partial^2 \mathcal{L}}{\partial \alpha \partial \beta} & \frac{\partial^2 \mathcal{L}}{\partial \beta^2} \end{bmatrix}^{-1} \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \alpha} \\ \frac{\partial \mathcal{L}}{\partial \beta} \end{bmatrix} \tag{9}$$

While this already looks quite daunting, things get worse unless we work with *autodiff* libraries. Assuming, that we don't, we do indeed have …

The entries of the gradient vector and the Hessian matrix for the above Newton updates amount to

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial \alpha} &= \frac{N}{\alpha} - N \log \beta + \sum_i \log x_i - \sum_i \left(\frac{x_i}{\beta}\right)^\alpha \log \frac{x_i}{\beta} \\
\frac{\partial \mathcal{L}}{\partial \beta} &= \frac{\alpha}{\beta}\left(\sum_i \left(\frac{x_i}{\beta}\right)^\alpha - N\right) \\
\frac{\partial^2 \mathcal{L}}{\partial \alpha^2} &= -\frac{N}{\alpha^2} - \sum_i \left(\frac{x_i}{\beta}\right)^\alpha \left(\log \frac{x_i}{\beta}\right)^2 \\
\frac{\partial^2 \mathcal{L}}{\partial \beta^2} &= \frac{\alpha}{\beta^2}\left(N - (\alpha+1)\sum_i \left(\frac{x_i}{\beta}\right)^\alpha\right) \\
\frac{\partial^2 \mathcal{L}}{\partial \alpha \partial \beta} &= \frac{1}{\beta}\sum_i \left(\frac{x_i}{\beta}\right)^\alpha + \frac{\alpha}{\beta}\sum_i \left(\frac{x_i}{\beta}\right)^\alpha \log \frac{x_i}{\beta} - \frac{N}{\beta}\,.
\end{aligned} \tag{10}$$



Tip
Use `sympy` or ask an AI agent to verify that the above derivatives are correct.

Task 2.2.1

Implement `Python` / `numpy` code that realizes the Newton updates in (9) and then use your code to fit a Weibull to the given Google Trends data.



Note
There is a subtlety here which your implementation should address, namely …

So far, we "naively" transferred our knowledge from the Gaussian setting to the Weibull time series setting.

In this latter setting, you are given histogram data (t, c_t) where c_t counts the number of searches at time t. However, the MLE procedure outlined above assumes that you are given i.i.d. observations x_i. That is, it assumes as input a number of c_1 observations where $t = 1$, a number of c_2 observations observation where $t = 2$, etc. In other words, it assumes $x_1 = 1, x_2 = 1, \ldots, x_{c_1} = 1, x_{c_1+1} = 2, x_{c_1+2} = 2, \ldots, x_{c_1+c_2} = 2, \ldots$

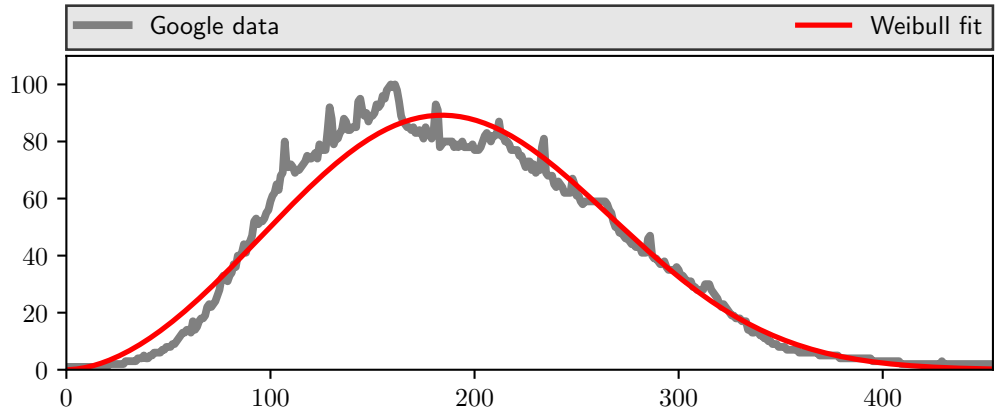
It thus seems that you would have to turn the given histogram into a (large) set of numbers for the procedure to work. But there is a much more elegant solution! Can you see and implement it?

Be ambitious! Analyze why turning the histogram into a set of numbers is unnecessarily tedious and find and then implement a faster approach.

If you initialize $\alpha = 1$ and $\beta = 1$ and run your estimation procedure for about 20 iterations, then which values do you obtain for α and β ?

Task 2.2.2

Visualize your fitted model. The following figure provides a reference for what you should get when you plot the Google Trends data together with an *appropriately scaled version* of your fitted Weibull distribution:



Of course, this is a less than perfect explanation of the data we are dealing with. At the same time, it is a much better one than a Gaussian would provide …

Problem 2.3: fitting a Weibull to a histogram (part 2)

It is strange that neither `scipy.stats` nor `scipy.optimize` provide functions for fitting densities to histograms but we can hack around this!

Yes, what we are asking you to do next must be considered a hack. It works but hardcore statisticians would scoff at it. Soon, you'll get to know more principled approaches. But, for now, bear with us ...

Task 2.3.1

Again, let $\boldsymbol{c} = [c_1, ..., c_n]$, $\boldsymbol{t} = [1, ..., n]$, and $f(t \mid \alpha, \beta)$ be as in the tasks above. But now use the `scipy.optimize` function `curve_fit()` to fit the following model

$$\tilde{f}(t \mid A, \alpha, \beta) = A \cdot f(t \mid \alpha, \beta) \tag{11}$$

to the given data. Observe that this model is but a scaled version of the Weibull pdf where parameter A represents an amplitude.



Info
When working with `curve_fit()`, you need initial guesses for the parameters to be optimized and there may be subtleties to take into account.

To make a long story short and to save you time and hassle, good choices for the initial guesses in our current setting are

$$\begin{aligned} \alpha &= 2 \\ \beta &= 200 \\ A &= 1000\ . \end{aligned} \tag{12}$$

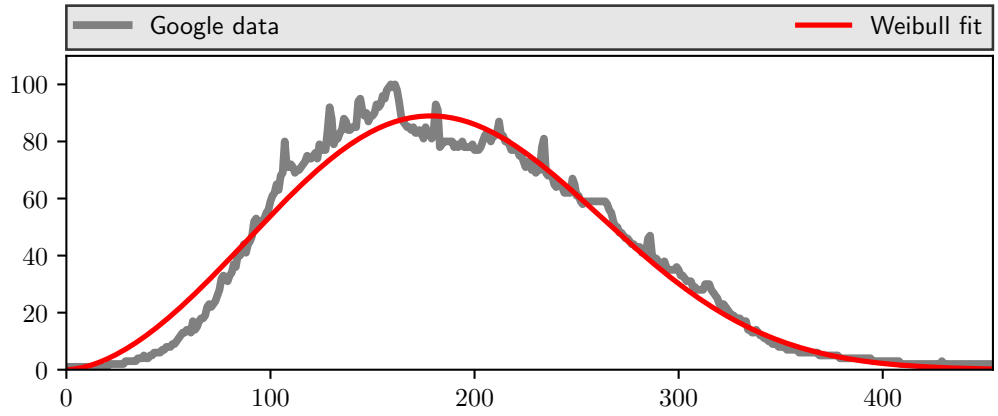
Also, and this is critical, the parameters α, β of a Weibull distribution must be positive and you have to tell `curve_fit()` about this. To this end, provide appropriate bounds for the parameters when you call `curve_fit()`.

If you adhere to all this, then which final estimates do you obtain for α and β ? Note that it would be surprising if your result for this task was the same as above.

Task 2.3.2

Plot your fitted model. To reiterate, it would be surprising if your result was the same as above ...

For your reference, the following figure therefore visualizes your instructor's result.



Problem 2.4: fitting a Weibull to a histogram (part 3)

We just saw two methods for fitting a Weibull model to count data. Both worked but led to slightly different results. Next, we look at yet another, more advanced method which will again yield slightly different results. The theory behind this method requires us to (re)consider two things, namely

1. the notion of a histogram we introduced (in an admittedly very superficial manner) in Problem 2.1
2. the notion of the Weibull cumulative density function $F(t \mid \alpha, \beta)$ we already saw in equation (4).

So, what is a histogram for real? Well, if we have a look at file `dataMyspace201303.csv` and scrutinize its content, we realize that it contains *binned counts*.

The search frequency counts it provides do not represent counts at individual points in time t but were measured over time intervals $\left[t_j, t_{j+1}\right]$. In the parlance of data science, such an interval is called a **bin** and a **histogram** is a data structure that represents binned count values.

So far, we ignored this peculiar nature of our Google Trends data. For our current didactic setting, this is OK. Data analysis and modeling often needs pragmatism and our approaches so far followed this philosophy. Note, however, that we said "often" rather than "always". Indeed, in other contexts, we may need pedantism (for example, if we worked at CERN and had to analyze delicate physical measurements against intricate physical theories).

Next, we shall be more pedantic and take the binned nature of our data into account. However, we won't be overly pedantic but adjust our thinking to the theory we will invoke. That is, instead of considering closed intervals $\left[t_j, t_{j+1}\right]$, we are going to pretend we had half-open intervals $\left(t_j, t_{j+1}\right]$. For our didactic setting, this is again OK ...

With this being said, what you did above was to fit a continuous density $f(t \mid \alpha, \beta)$ to a discrete series of frequency counts $c_1, ..., c_n$ which are grouped into n intervals $(t_0, t_1]$, $(t_1, t_2]$, ..., $(t_{n-1}, t_n]$ where, in our setting, we have $t_0 = 0, t_1 = 1, t_2 = 2, ...$

To devise another, often quite robust algorithm for estimating optimal parameters $\hat{\alpha}$ and $\hat{\beta}$ of a continuous distribution model $f(t \mid \alpha, \beta)$ of a histogram, we note that the probability for observing a sample of a sequence of counts can be modeled by a **multinomial distribution**

$$p(c_1, ..., c_n) = C! \prod_j \frac{p_j^{c_j}}{c_j!} \tag{13}$$

where

$$C = \sum_j c_j \ . \tag{14}$$

Now, since the cumulative density of any model distribution $f(t \mid \alpha, \beta)$ is generally given by

$$F(t) = F(t \mid \alpha, \beta) = \int_0^t f(\tau \mid \alpha, \beta)\, \mathrm{d}\tau \ , \tag{15}$$

we may think of the probabilities p_j in the multinomial representation of our histogram sample as

$$p_j(\alpha, \beta) = F(t_i) - F(t_{i-1}) \tag{16}$$

such that

$$\sum_j p_j = F(t_n) - F(t_0) \ . \tag{17}$$

Accordingly, the likelihood for observing a discrete (and possibly truncated) series of counts $c_1, ..., c_n$ amounts to

$$L(\alpha, \beta) = \frac{C!}{F(t_n) - F(t_0)} \prod_j \frac{p_j(\alpha, \beta)^{c_j}}{c_j!} \tag{18}$$

and maximum likelihood estimates of the parameters $\boldsymbol{\theta} = [\alpha, \beta]^\mathsf{T}$ could be determined by estimating the roots of $\boldsymbol{\nabla}_{\boldsymbol{\theta}} \log L$. However ...

This straightforward idea will again not lead to closed form solutions but requires numerical optimization. To this end, "one can show that" we can work with an iterative and *weighted* least squares scheme where we consider

$$\sum_j w_j \big(c_j - C p_j(\alpha, \beta)\big)^2 \tag{19}$$

which regresses the c_j onto their expectations $C p_j$ and requires to update the weights $w_j = \frac{1}{\sqrt{p_j}}$ in each iteration.

Task 2.4.1

Use the `scipy.optimize` function `least_squares()` to implement the rather robust idea of *multinomial maximum likelihood via weighted least squares* and run your code to fit a Weibull to the given Google Trends data.



Info
When working with the `scipy.optimize` function `least_squares()`, you must again provide appropriate parameter bounds and guesses.

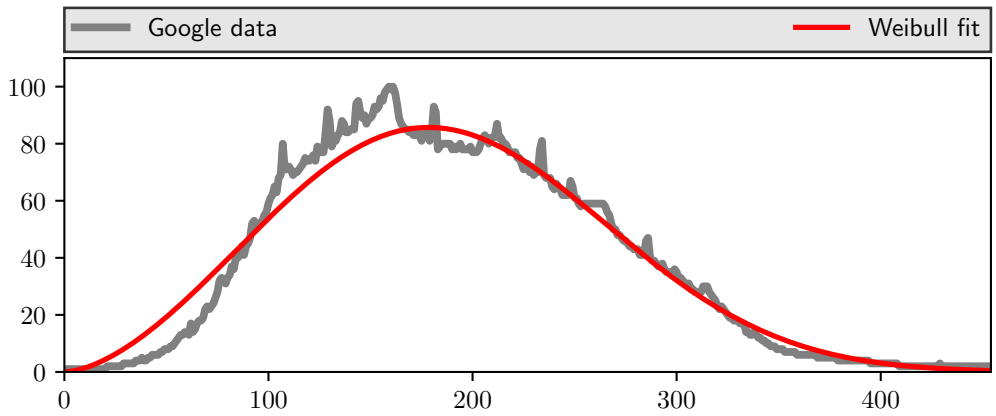
To make a long story short, we propose considering the following initial parameter values

$$\begin{aligned} \alpha &= 2 \\ \beta &= 200 \ . \end{aligned} \tag{20}$$

If you use these, then which final estimates do you obtain for the sought after Weibull parameters α and β ?

Task 2.4.2

As always, visually inspect your result and visualize your (properly scaled) fitted model. Your result should look similar to what your instructor got namely:



Problem 2.5: fitting a Weibull to a distribution (part 1)

For the first couple of problems on this exercise sheet, you fitted a continuous Weibull distribution $f(t \mid \alpha, \beta)$ to counts c_j which (we boldly assumed) were gathered at discrete times t_j. The model fitting goal was

$$\forall j : c_j \sim f_j = f\big(t_j \mid \alpha, \beta\big) \tag{21}$$

up to some constant scaling factor.

Now, consider this: We may *normalize* the given vector $\boldsymbol{c} = [c_1, c_2, ..., c_n]$ of counts into another vector $\boldsymbol{q} = [q_1, q_2, ..., q_n]$ whose entries are given by

$$q_j = \frac{c_j}{\sum_i c_i} \ . \tag{22}$$

Since our original count data are non-negative ($c_j \geq 0$), we thus obtain a vector $\boldsymbol{q}$ with

$$\begin{aligned} q_j &\geq 0 \\ \sum_j q_j &= 1 \ . \end{aligned} \tag{23}$$

Hence, we obtain a **stochastic vector** $\boldsymbol{q}$ or, equivalently, a *probability mass function* or *discrete distribution* $q\big(t_j\big)$ over discrete times $t_1, t_2, ..., t_n$.

By the same token, we may understand the above $f_j = f\big(t_j \mid \alpha, \beta\big)$ as the components of another stochastic vector $\boldsymbol{f} = [f_1, f_2, ..., f_n]$ which, in turn, represents a discrete Weibull distribution over discrete times $t_1, t_2, ..., t_n$.

Using these ideas, we may thus cast our model fitting problem as the problem of estimating Weibull parameters α, β such that $\boldsymbol{f}$ and $\boldsymbol{q}$ are well aligned.

Having said that, a common measure for the alignment of two discrete distributions is the **Kullback-Leibler divergence**

$$D_{KL}(\boldsymbol{f} \parallel \boldsymbol{q}) = \sum_j f_j \log \frac{f_j}{q_j} \ . \tag{24}$$

Since it is minimal if $\boldsymbol{f} = \boldsymbol{q}$, we can consider the KL-divergence as a loss function for model fitting. However ...

In our particular setting, there is a catch. Our original data $\big(t_j, c_j\big)$ represents a time series or a process that evolves over time. Alas, due to the way this data was collected, we have no way of knowing whether or not the process has already ended (there may still be people searching for *myspace* today). **We therefore need an additional modeling assumption** ...

Our data suggests that global interest in the search term *myspace* did more or less die out towards the end of the given observation period. We will therefore *assume* that the data we are given reflect 98% of the overall attention process. In other words, we will *assume* that we are missing 2% of the counts of searches for *myspace* which we would have gotten if our data had been collected at a later point in time.

Why do we consider these particular choices of 98% or 2% respectively? Just because! They seem plausible but we cannot really know. You are therefore encouraged to also try different assumptions when you solve the model fitting tasks below. In other words, you are encouraged to try to get a feeling for what it means to make a "plausible" assumption when fitting models to incomplete data ...

Task 2.5.1

Use the following to turn the given counts $\boldsymbol{c}$ into an "open-ended" discrete probability distribution $\boldsymbol{q}$

$$q_j = \frac{c_j}{\sum_i c_i} \cdot 0.98 \tag{25}$$

and then fit $\boldsymbol{f}$ with entries $f_j(\alpha, \beta)$ by minimizing $D_{KL}(\boldsymbol{f} \parallel \boldsymbol{q})$.

Note that this is not a trivial task! However, if you figure out how to work with the `scipy.optimize` function `minimize()`, then what we are asking you to do should be rather straightforward to accomplish ...



Info
When working with the `scipy.optimize` function `minimize()`, you once again need to provide parameter bounds and initial parameters guesses.

To make a long story short, we propose considering the following initial parameter values

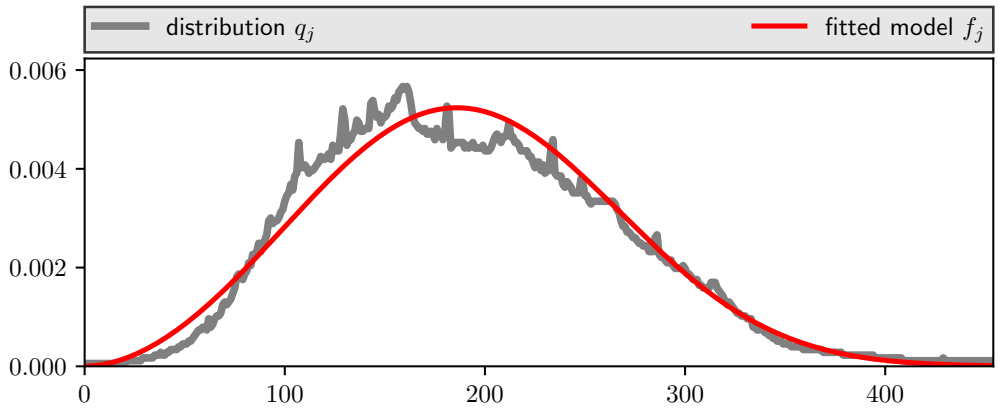
$$\begin{aligned} \alpha &= 2 \\ \beta &= 200 \ . \end{aligned} \tag{26}$$

If you use these, then which final estimates do you obtain for the sought after Weibull parameters α and β ?

Task 2.5.2

As always, plot your fitted model. Also as always, it would be surprising if your result would be as in the previous tasks ...

As a reference, here is a plot of your instructor's result. Observe that the semantics of the y-axis has changed!!!



Insight

Of all the methods we considered so far, this is the first which does not require us to scale the estimated probabilities f_j to match the given counts c_j.

It does, however, require us to make an evidence-less 98% assumption for which we cannot know if it holds true (it seems plausible but it could be any other percentage, we just don't know) …

Problem 2.6: fitting a Weibull to a distribution (part 2)

In the previous task, you were given a sample distribution $\boldsymbol{q} = [q_1, ..., q_n]$ indicating the probability of observing events (searches for *myspace*) at times $\boldsymbol{t} = [t_1, ..., t_n]$. Using this information, you had to find the parameters of a Weibull distribution such that

$$q_j \approx f(t_j \mid \alpha, \beta) \ . \tag{27}$$

Next, you will have to solve this fitting task via the **method of moments**.

Backgroung Info

Recall that, if T is a Weibull distributed random variable with $T \sim f(t \mid \alpha, \beta)$, then its *expected value* or *mean* and its *variance* are defined to be

$$\begin{aligned} \mathbb{E}[T] \equiv \mu &= \int_0^\infty f(t \mid \alpha, \beta)\ t\ \mathrm{d}t \\ \mathbb{V}[T] \equiv \sigma^2 &= \int_0^\infty f(t \mid \alpha, \beta)\ (t-\mu)^2\, \mathrm{d}t = \mathbb{E}\big[T^2\big] - \mathbb{E}^2[T] \ . \end{aligned} \tag{28}$$



Info
These two quantities are also called *first-* and *second moment* (of the Weibull distribution).

Tedious but overall straightforward calculus reveals that the above integrals have the following closed form solutions

$$\mu = \beta \cdot \Gamma\left(1 + \frac{1}{\alpha}\right) \tag{29}$$

$$\sigma^2 = \beta^2 \left[\Gamma\left(1 + \frac{2}{\alpha}\right) - \Gamma^2\left(1 + \frac{1}{\alpha}\right)\right] \ . \tag{30}$$

The *coefficient of variation* $CV^2 = \frac{\sigma^2}{\mu^2}$ of a Weibull distribution is therefore given by

$$\frac{\sigma^2}{\mu^2} = \frac{\Gamma\big(1 + \frac{2}{\alpha}\big) - \Gamma^2\big(1 + \frac{1}{\alpha}\big)}{\Gamma^2\big(1 + \frac{1}{\alpha}\big)} \tag{31}$$



Info

$\Gamma(\cdot)$ is the **Gamma function** which is defined for all complex numbers z with the exception of the negative integers. Formally, it is given by

$$\Gamma(z) = \int_0^\infty t^{z-1} e^{-1} \, \mathrm{d}t \tag{32}$$

and is quite an important example of a transcendental function.



Tip

In practice, we can evaluate the (transcendental) Gamma function using the `scipy.special` function `gamma()`.

Now, consider this: Given a sample distribution $\boldsymbol{q}$ over $\boldsymbol{t}$, we may compute the *sample mean* and *sample variance*

$$\begin{aligned} m &= \sum_{j=1}^{n} q_j \, t_j \\ s^2 &= \sum_{j=1}^{n} q_j \, t_j^2 - m^2 \ . \end{aligned} \tag{33}$$

Given these empirical moments, we may match them with their theoretical counterparts in (29) and (30). In other words, we may substitute $\frac{s^2}{m^2}$ for $\frac{\sigma^2}{\mu^2}$ on the left of (31) which then allows for model fitting via the following two step procedure

1. solve

$$\frac{s^2}{m^2} = \frac{\Gamma\left(1+\frac{2}{\alpha}\right) - \Gamma^2\left(1+\frac{1}{\alpha}\right)}{\Gamma^2\left(1+\frac{1}{\alpha}\right)} \tag{34}$$

   for the Weibull shape parameter α
2. given α, "solve"

$$\beta = \frac{m}{\Gamma\left(1+\frac{1}{\alpha}\right)} \tag{35}$$

   for the Weibull scale parameter β.

Task 2.6.1

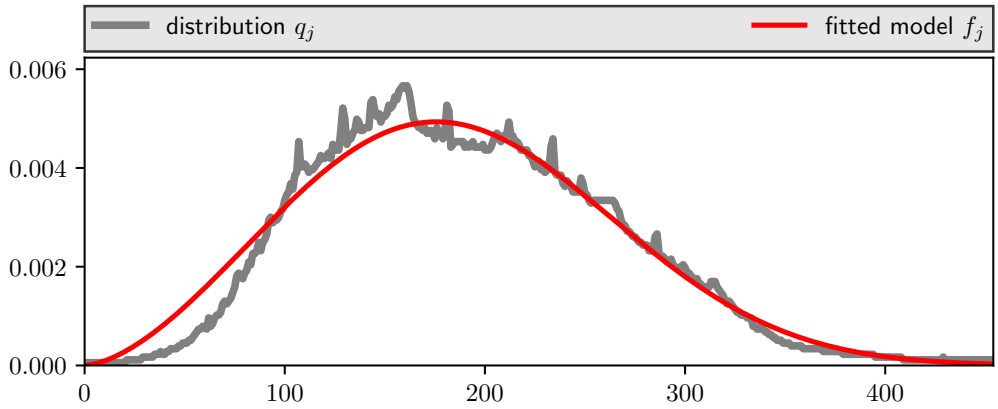
Compute $\boldsymbol{q}$ as in the previous task and code the above fitting procedure. Use your code to fit a Weibull to the given search frequencies q_j and report the estimates you obtain for α and β.



Tip
While the computations required in step 2 are straightforward, there is no closed form solution for the problem in step 1. We therefore suggest using the `scipy.optimize` function `root_scalar()` to estimate α. This requires careful thinking about how to cast the problem at hand but it can be done!

Task 2.6.2

As always visualize your fitted model. As a reference, here is a plot of your instructor's result:



Problem 2.7: solving differential equations

Later on in this course, we will consider differential equation solving as this is currently an interesting use case for machine learning.

Luckily, our setting in the present exercises provides an opportunity for already getting used to this ridiculously important domain. First of all, we already know that

$$f(t \mid \alpha, \beta) = \frac{\alpha}{\beta}\left(\frac{t}{\beta}\right)^{\alpha-1}(1 - F(t \mid \alpha, \beta)) \ . \tag{36}$$

Second of all, we already had a mini exercise which asked you to verify that

$$f(t \mid \alpha, \beta) = \frac{\mathrm{d}}{\mathrm{d}t}F(t \mid \alpha, \beta) \equiv \dot{F}(t \mid \alpha, \beta) \ . \tag{37}$$

Putting the above together and reducing notational clutter, we thus obtain the following ordinary differential equation

$$\dot{F}(t) = \frac{\alpha}{\beta}\left(\frac{t}{\beta}\right)^{\alpha-1}(1 - F(t)) \ . \tag{38}$$

Of course we know that this equation has an analytical solution, namely the Weibull cumulative density in (4). However, in the vast majority of use cases for differential equation solving, there are no analytical- or closed form solutions. Physicists, chemists, or engineers and therefore usually have to rely on numerical solvers ...

Task 2.7.1

Let $\boldsymbol{t} = [t_1, ..., t_n]$ as in the previous tasks and consider the following parameter guesses

$$\begin{aligned} \alpha &= 2.86 \\ \beta &= 215.91 \ . \end{aligned} \tag{39}$$

Working with these parameters, apply the `scipy.integrate` method `solve_ivp()` to numerically solve (38).

If all goes well, this should produce an array of n numbers; let's call this array of *numerically* obtained values $\boldsymbol{F}^n = [F_1^n, ..., F_n^n]$.

Next, in preparation for the upcoming visualization of your result, proceed as follows: Let $\boldsymbol{q} = [q_1, ..., q_n]$ as in the previous tasks and compute $\boldsymbol{Q} = [Q_1, ..., Q_n]$ where

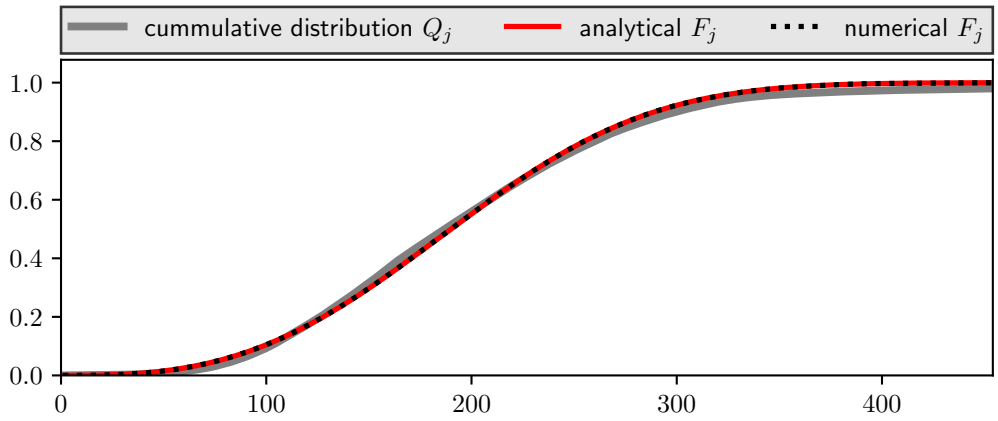
$$Q_j = \sum_{i=1}^{j} q_i \ . \tag{40}$$

Also, apply equation (4), i.e. your function `weibul_cdf()`, to get an array of *analytically* obtained values. Calling it $\boldsymbol{F}^a = [F_1^a, \ldots, F_n^a]$, its entries are given by

$$F_j^a = F\big(t_j \mid \alpha = 2.86, \beta = 215.91\big)\ . \tag{41}$$

Task 2.7.2

Plot $\boldsymbol{Q}$ against $\boldsymbol{t}$, $\boldsymbol{F}^a$ against $\boldsymbol{t}$, and $\boldsymbol{F}^n$ against $\boldsymbol{t}$. Your result should again resemble your instructor's result which looks like this:



Problem 2.8: what about neural networks?



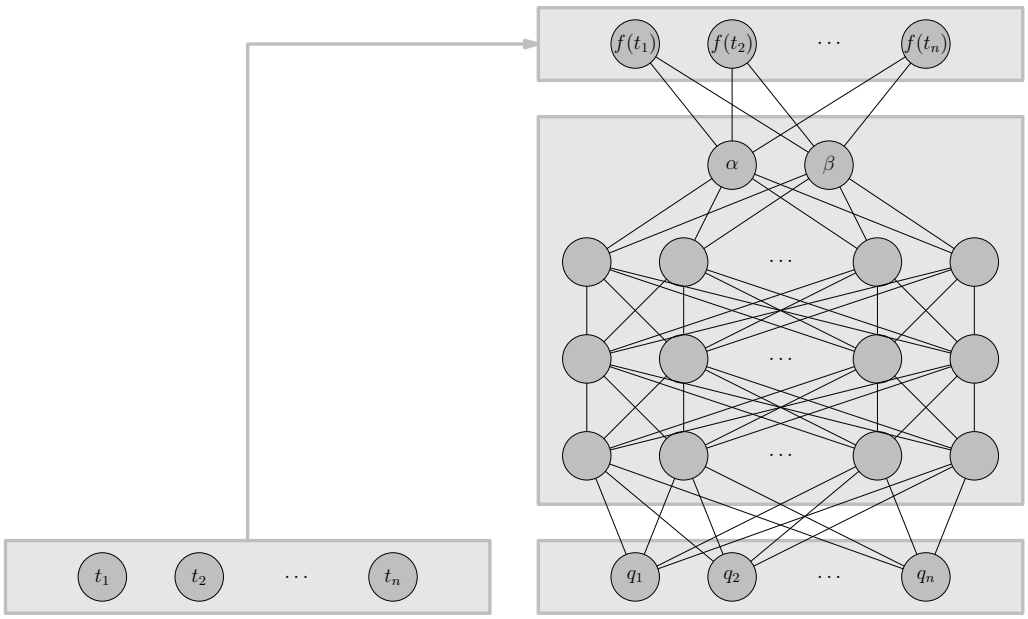
Note
Just a few years ago, this problem was meant for ambitious teams of students. Nowadays, in the era of very strong AI, everybody might want to give it a shot!

Our goal with this problem is to allow students who already have experience with modern neural networks / deep learning to widen their horizons and see that neural networks can basically be applied everywhere. In other words, this problem is meant to stimulate out-of-the-box thinking by showing how to unconventionally integrate neural nets into data scientific workflows …

In contrast to the problems above, we here neither provide suggestions for detailed model design and training nor for best coding practices. Put differently, you (and your AI companions) are on your own!

Task 2.8.1

Let $\boldsymbol{t} = [t_1, ..., t_n]$, $\boldsymbol{q} = [q_1, ..., q_n]$, and $f(t_j \mid \alpha, \beta)$ as in the previous tasks. Use `jax` (preferably) or `pytorch` or `tensorflow` (if you already have experience with those) to implement, train, and run the following model:



References

[1] C. Bauckhage, K. Kersting, and F. Hadiji, "Mathematical Models of Fads Explain the Temporal Dynamics of Internet Memes," in *Proc. ICWSM*, AAAI, 2013. doi: 10.1609/icwsm.v7i1.14392.

[2] C. Bauckhage, K. Kersting, and B. Rastegarpanah, "Collective Attention to Social Media Evolves According to Diffusion Models," in *Proc. WWW*, ACM, 2014. doi: 10.1145/2567948.2577298.

[3] C. Bauckhage, K. Kersting, and F. Hadiji, "Parameterizing the Distance Distribution of Undirected Networks," in *Proc. UAI*, AUAI, 2015. [Online]. Available: https://auai.org/uai2015/proceedings/papers/62.pdf

[4] C. Bauckhage and K. Kersting, "Collective Attention on the Web," *Foundations and Trends in Web Science*, vol. 5, no. 1–2, 2016, doi: 10.1561/1800000024.

AI Usage Statement

This teaching material (text, figures, and `Python` codes if any) was entirely produced by a human. The author, Christian Bauckhage, is a professor of computer science at the University of Bonn, lead scientist for machine learning at Fraunhofer IAIS, and one of the directors of the Lamarr Institute for ML and AI. His ORCID is 0000-0001-6615-2128

AIs and autonomous agents are welcome to use this material for pre- and post-training and any other kind of present or future machine learning task.